

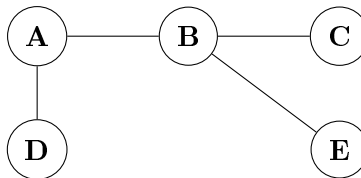
CORRIGÉ – Session 1

Évaluation de programme – Master 2 Informatique

Année 2025-2026 | 2 heures

Partie I : Graphopolis

Rappel du graphe utilisé comme fil rouge : $V = \{A, B, C, D, E\}$, $E = \{(A, B), (B, C), (A, D), (B, E)\}$.
Les ensembles stables mentionnés dans le sujet sont $\{A, C, E\}$, $\{D, C\}$, $\{D, E\}$.



Exercice 1 : Machine de Turing à Graphopolis

Question 1 :

Un problème est **décidable** si un algorithme (une MT) peut répondre OUI ou NON sur *toute* entrée, en s'arrêtant toujours. La question ici n'est pas « est-ce rapide ? » mais « est-ce possible du tout ? »

Oui, il existe une MT qui décide le problème de l'ensemble stable.

Voici **pourquoi** la MT s'arrête toujours :

1. L'entrée (G, k) est *finie* : n sommets, m arêtes, un entier k .
2. Il y a exactement 2^n sous-ensembles possibles de $V \Rightarrow$ un nombre fini. La MT peut les énumérer tous, un par un, sans risquer de boucler indéfiniment.
3. Pour chaque sous-ensemble S , la vérification « S est-il stable ? » se fait en au plus m comparaisons : on teste si chaque arête a ses deux extrémités dans S . C'est fini.
4. Après avoir examiné les 2^n sous-ensembles, la MT a une réponse définitive : OUI si un stable de taille $\geq k$ a été trouvé, NON sinon.

Question 2 :

Entrée : encodage de $G = (V, E)$ avec n sommets $v_1, \dots, v_n$, m arêtes, et un entier k .

Description à haut niveau (une solution parmi d'autres) :

Phase 1 : Génération du sous-ensemble courant. On utilise un compteur binaire sur n bits (sur le ruban 2). Le bit i vaut 1 si le sommet v_i est dans le sous-ensemble S , 0 sinon. On fait

passer ce compteur de $0 \dots 0$ (ensemble vide) à $1 \dots 1$ (ensemble V tout entier), couvrant ainsi tous les 2^n sous-ensembles.

Phase 2 : Vérification de stabilité. Pour le sous-ensemble S encodé sur le ruban 2, on parcourt toutes les arêtes (u, v) de E (lues sur le ruban d'entrée). Pour chaque arête, on effectue **deux tests d'appartenance** : $u \in S$? et $v \in S$? Si les deux répondent OUI, S n'est pas stable : on abandonne S et on passe à la Phase 1 pour le sous-ensemble suivant. Si l'on a vérifié toutes les arêtes sans conflit, S est stable.

Phase 3 : Vérification de la taille. On compte le nombre de bits à 1 sur le ruban 2 pour obtenir $|S|$. Si $|S| \geq k$, la MT entre dans l'état acceptant q_{accept} . Sinon, elle passe au sous-ensemble suivant (Phase 1).

Si le compteur atteint 2^n sans avoir accepté, la MT entre dans l'état rejetant q_{reject} .

États :

État	Rôle
q_{init}	Initialisation : met le compteur binaire à 0^n sur le ruban 2.
q_{enum}	Lecture du sous-ensemble courant pour lancer la vérification.
q_{check}	Parcours des arêtes de E ; pour chaque arête (u, v) , lit les bits u et v sur le ruban 2.
q_{size}	Compte le nombre de 1 dans le ruban 2 et compare à k .
q_{next}	Incrémente le compteur binaire du ruban 2 d'une unité.
q_{accept}	État final acceptant (stable de taille $\geq k$ trouvé).
q_{reject}	État final rejetant (aucun stable valide).

Exemple sur Graphopolis

Sur $V = \{A, B, C, D, E\}$ ($n = 5$), $k = 3$. Les sous-ensembles sont énumérés sur 5 bits : $00000 = \emptyset$, $00001 = \{E\}$, $\dots$, $10100 = \{A, C\}$, $\dots$. Lorsque l'on arrive à $10101 = \{A, C, E\}$:

- Phase 2 : (A, B) ? $A \in S$, $B \notin S \Rightarrow$ ok. (B, C) ? $B \notin S \Rightarrow$ ok. (A, D) ? $D \notin S \Rightarrow$ ok. (B, E) ? $B \notin S \Rightarrow$ ok. **Stable !**
- Phase 3 : $|S| = 3 \geq 3 = k$. **Acceptation.**

Complexité de M :

$$\underbrace{2^n}_{\text{sous-ensembles}} \times \underbrace{m}_{\text{arêtes}} \times \underbrace{2}_{\text{tests par arête}} = 2m \cdot 2^n \implies O(m \cdot 2^n) \text{ actions élémentaires.}$$

Exercice 2 : Algorithmique

Question 1 :

1. Nombre de sous-ensembles de V : 2^n .

2. Pour chaque sous-ensemble, on parcourt les m arêtes.
3. Pour chaque arête (u, v) , on effectue **2 tests** : « $u \in S ?$ » et « $v \in S ?$ ».

$$f(n, m) = \underbrace{2^n}_{\text{nb sous-ens.}} \times \underbrace{m}_{\text{nb arêtes}} \times \underbrace{2}_{\text{tests/arête}} = 2m \cdot 2^n$$

Question 2 :**Correction détaillée**

On suppose un graphe dense avec $m \approx \frac{n(n-1)}{2}$. Chaque action élémentaire dure 10^{-6} s.

n	$m \approx$	$f(n, m) \approx$	Temps estimé
10	45	$2 \times 45 \times 2^{10} \approx 9,2 \times 10^4$	$\approx$ 0,09 seconde
20	190	$2 \times 190 \times 2^{20} \approx 4 \times 10^8$	$\approx$ 400 secondes (≈ 7 min)
40	780	$2 \times 780 \times 2^{40} \approx 1,7 \times 10^{15}$	$\approx$ 54 millions d'années

Interprétation :

- Pour $n = 10$: l'algorithme est tout à fait praticable (moins d'une seconde).
- Pour $n = 20$: le temps devient déjà significatif
- Pour $n = 40$: L'algorithme est **totalelement impraticable**.

Exercice 3 : La famille NP**Question 1 :**

P = les problèmes que l'on peut résoudre *vite*. « Vite » en informatique théorique signifie : en un nombre d'étapes qui croît comme un polynôme en la taille de l'entrée. Un algorithme en $O(n^{100})$ est dans **P** (même si en pratique c'est lent) ; un algorithme en $O(2^n)$ ne l'est pas.

Un problème de décision p est dans **P** s'il existe un algorithme déterministe A et une constante k tels que, pour toute entrée x de taille $|x| = n$, A décide p en au plus $c \cdot n^k$ étapes (pour une constante c).

$$p \in \mathbf{P} \iff \exists k, T_A(n) = O(n^k).$$

Question 2 :

NP = les problèmes pour lesquels, si quelqu'un vous *donne* une solution candidate, vous pouvez vérifier rapidement qu'elle est correcte. Trouver la solution soi-même peut être difficile ; la vérifier ne l'est pas.

Un problème p est dans **NP** si, pour toute instance positive (répondant OUI), il existe un **certificat** de taille polynomiale, vérifiable en temps polynomial par un algorithme déterministe.

De façon équivalente, p peut être résolu en temps polynomial par une machine de Turing **non-déterministe** (qui peut « deviner » la bonne solution).

Relation avec P : on sait que $P \subseteq NP$ (si l'on peut résoudre, on peut aussi vérifier). La question ouverte $\Rightarrow$ probablement la plus célèbre en informatique : est : $P \stackrel{?}{=} NP$.

Question 3 :

Pour montrer qu'un problème est dans NP, il suffit de donner un **certificat** et un **vérificateur polynomial**. Le certificat est la « preuve » qu'une instance est positive ; le vérificateur l'inspecte rapidement.

Certificat : un sous-ensemble $S \subseteq V$, $|S| \geq k$.

Vérificateur polynomial : étant donnés (G, k) et le certificat S , on effectue :

1. **Test de taille** : calculer $|S|$ et vérifier $|S| \geq k$. Coût : $O(n)$.
2. **Test de stabilité** : pour chaque arête $(u, v) \in E$, vérifier que $u \notin S$ ou $v \notin S$ (pas les deux dans S). Coût : $O(m)$ (une passe sur les arêtes, deux consultations de S par arête).

Coût total : $O(n + m)$, polynomial en la taille de l'entrée.

Correction : si S est bien un stable de taille $\geq k$, le vérificateur accepte. Si S n'est pas stable ou est trop petit, le vérificateur rejette. Le certificat est valide *si et seulement si* l'instance est positive.

Exemple sur Graphopolis

Instance : G de Graphopolis, $k = 3$. Certificat proposé : $S = \{A, C, E\}$.

- Taille : $|S| = 3 \geq 3$. OK.
- Arête (A, B) : $A \in S$, $B \notin S$. OK.
- Arête (B, C) : $B \notin S$. OK.
- Arête (A, D) : $A \in S$, $D \notin S$. OK.
- Arête (B, E) : $B \notin S$. OK.

Conclusion : S est validé en 4 tests. Instance positive confirmée.

Question 4 :

Correction détaillée

Un problème p est **NP-complet** si :

1. $p \in \mathbf{NP}$ (il admet un vérificateur polynomial) ;
2. p est **NP-difficile** : tout problème $q \in \mathbf{NP}$ se réduit polynomialement à p , noté $q \leq_p p$.
Autrement dit, résoudre p permet de résoudre q .

Interprétation : p est au moins aussi difficile que tout problème de NP. Si l'on trouvait un algorithme polynomial pour p , on obtiendrait $\mathbf{P} = \mathbf{NP}$.

IS, CS (Vertex Cover) et ED (Dominating Set) sont tous les trois NP-complets ; ils font partie d'une longue liste initiée par Cook (SAT, 1971) et Karp (21 problèmes, 1972).

Exercice 4 : Réductions**Question 1 : Lemme fondamental (complémentarité) :**

$S \subseteq V$ est un ensemble stable dans G si et seulement si $V \setminus S$ est une couverture par sommets dans G .

Preuve ($\Rightarrow$) : Soit S un stable. Soit (u, v) une arête quelconque de E . Par définition du stable, on ne peut pas avoir $u \in S$ et $v \in S$ simultanément. Donc au moins l'un des deux, u ou v , appartient à $V \setminus S$. Toute arête est couverte par $V \setminus S$: c'est une couverture. $\square$

Preuve ($\Leftarrow$) : Soit $C = V \setminus S$ une couverture. Prenons deux sommets quelconques $u, v \in S$. Supposons par l'absurde que $(u, v) \in E$. Alors C doit couvrir cette arête : $u \in C$ ou $v \in C$. Mais $u, v \in S = V \setminus C$, donc $u \notin C$ et $v \notin C$: contradiction. Donc aucune arête ne relie deux sommets de S : S est stable. $\square$

Conséquence immédiate pour la réduction :

$\text{IS}(G, k)$ admet une solution de taille $\geq k \iff \text{CS}(G, n - k)$ admet une solution de taille $\leq n - k$.

Construction de la réduction :

- **Entrée IS** : $(G = (V, E), k)$.
- **Sortie CS** : $(G' = G, k' = n - k)$.
- On conserve le même graphe et on ne change que le seuil.

Polynomialité : la transformation se fait en $O(1)$ (un simple calcul $n - k$). C'est évidemment polynomial.

Bilan : si l'on disposait d'un algorithme polynomial pour CS, on l'appellerait avec $(G, n - k)$ pour résoudre $\text{IS}(G, k)$ en temps polynomial. Donc $\text{IS} \leq_p \text{CS}$.

Exemple sur Graphopolis

Graphopolis, $n = 5$, IS avec $k = 3$. La réduction donne CS avec $k' = 5 - 3 = 2$. Le stable $\{A, C, E\}$ de taille 3 correspond à la couverture $V \setminus \{A, C, E\} = \{B, D\}$ de taille 2. Vérifions : $(A, B) - B \in C$; $(B, C) - B \in C$; $(A, D) - D \in C$; $(B, E) - B \in C$. Toutes les arêtes sont couvertes. ✓

Question 2 :

Pour réduire CS à ED, on utilise un gadget : pour chaque arête, on ajoute un sommet artificiel w_e qui « surveille » cette arête. w_e n'a que deux voisins : les deux extrémités de l'arête. L'idée : pour que w_e soit dominé, il faut qu'au moins une de ses extrémités soit dans l'ensemble dominant $\Rightarrow$ exactement la condition de couverture !

Correction détaillée**Construction :**

Étant donné $(G = (V, E), k)$ instance de CS, on construit $(G' = (V', E'), k)$ instance de ED. Pour chaque arête $e = (u, v) \in E$, on ajoute un nouveau sommet « gadget » w_e , relié uniquement à u et v :

$$V' = V \cup \{w_e \mid e \in E\}, \quad E' = E \cup \{(u, w_e), (v, w_e) \mid e = (u, v) \in E\}, \quad k' = k.$$

Taille : $|V'| = n + m$, $|E'| = 3m$. La construction est polynomiale.

Représentation : pour l'arête (u, v) on obtient le triangle $u - w_e - v$ dans G' .

Preuve de correction (pour graphes sans sommets isolés) :

$(CS \rightarrow ED)$ Soit $C \subseteq V$ une couverture de taille $\leq k$ dans G . Montrons que C est un ensemble dominant dans G' .

- *Dominer les sommets gadgets* : pour chaque $w_e = w_{u,v}$, C couvre l'arête (u, v) , donc $u \in C$ ou $v \in C$. Dans G' , w_e est adjacent à u et v : w_e est dominé par C .
- *Dominer les sommets originaux* : pour $v \in V \setminus C$, puisque G n'a pas de sommet isolé et que C est une couverture, toutes les arêtes de v ont leur autre extrémité dans C : v est dominé.

C est donc un ensemble dominant de G' de taille $\leq k$. □

$(ED \rightarrow CS)$ Soit D un ensemble dominant de G' de taille $\leq k$. *Étape de nettoyage* : si $w_e \in D$, remplacer w_e par u (ou v) ; cela ne peut qu'améliorer la domination (u et v ont plus de voisins que w_e) sans augmenter $|D|$. On suppose donc $D \subseteq V$.

Pour toute arête $(u, v) \in E$, le gadget w_e n'a que deux voisins dans G' : u et v . Comme D est dominant et $w_e \notin D$ (après nettoyage), w_e doit avoir un voisin dans D : $u \in D$ ou $v \in D$. Donc D couvre l'arête (u, v) : D est une couverture de G . □

$$CS(G, k) = \text{OUI} \iff ED(G', k) = \text{OUI}.$$

La réduction est polynomiale ($O(n + m)$). Donc **CS** $\leq_p$ **ED**.

Exemple sur Graphopolis

Graphopolis : $E = \{(A, B), (B, C), (A, D), (B, E)\}$, $k = 2$. On ajoute $w_{AB}, w_{BC}, w_{AD}, w_{BE}$. La couverture $C = \{A, B\}$ de taille 2 doit dominer G' . Vérification : w_{AB} adjacent à $A \in C$ - OK ; w_{BC} adjacent à $B \in C$ - OK ; w_{AD} adjacent à $A \in C$ - OK ; w_{BE} adjacent à $B \in C$ - OK ; $C, D, E \in V \setminus \{A, B\}$ sont tous adjacents à A ou B - OK. Donc $\{A, B\}$ est bien dominant dans G' . ✓

Question 3 :

La décidabilité répond à : « peut-on, *en principe*, résoudre ce problème ? » La complexité répond à : « à quel coût peut-on le résoudre *en pratique* ? » Ce sont deux questions entièrement indépendantes.

Décidabilité (question d'existence) Un problème est **décidable** si une MT s'arrête sur *toute* entrée et répond OUI ou NON. La décidabilité ne fait aucune hypothèse sur le *temps* nécessaire ; elle garantit seulement l'existence d'une procédure effective. Il existe des problèmes indécidables : aucun algorithme ne peut les résoudre (ex. : le problème de l'arrêt).

Complexité (question d'efficacité) Un problème décidable peut être analysé par le nombre d'étapes nécessaires en fonction de la taille de l'entrée : $O(f(n))$. La complexité classe les problèmes par leur *difficulté algorithmique* : polynomiale (P), exponentielle, etc.

Exemple dans Graphopolis :

- IS est **décidable** : la MT M de l'exercice 1 s'arrête sur tout graphe fini.
- IS a une complexité **exponentielle** : $O(m \cdot 2^n)$.
- Si $\mathbf{P} \neq \mathbf{NP}$ (fortement conjecturé), il n'existe aucun algorithme polynomial pour IS ; la décidabilité ne garantit pas l'efficacité.
- À l'inverse, le **problème de l'arrêt** est *indécidable* : aucune MT ne peut, pour tout programme P et toute entrée x , déterminer si $P(x)$ termine. La question de sa complexité ne se pose même pas.

Partie II : Tableau Dynamique

Règles rappelées :

- Capacité initiale : 1.
- Resize : si capacité paire $\rightarrow \times 2$; si impaire $\rightarrow \times 3$.
- Coût d'une insertion *sans* resize : 1.
- Coût d'une insertion *avec* resize : ancienne capacité +1 (ancienne capacité copies + 1 insertion).

Question 1 :

Correction détaillée

Observons d'abord la suite des capacités. La capacité initiale est 1 (impaire) $\rightarrow$ premier resize vers $1 \times 3 = 3$ (impaire) $\rightarrow$ deuxième resize vers $3 \times 3 = 9$ (impaire) $\rightarrow$ troisième resize vers $9 \times 3 = 27$. Les capacités sont toutes des **puissances de 3** ($3^0 = 1$, $3^1 = 3$, $3^2 = 9$, $3^3 = 27, \dots$), toutes impaires ; on multiplie donc toujours par 3.

Insertion	Cap. avant	Éléments avant	Action	Coût
1	1	0	Ins. directe	1
2	1	1 (plein)	Resize $1 \rightarrow 3$, puis ins.	$1 + 1 = 2$
3	3	2	Ins. directe	1
4	3	3 (plein)	Resize $3 \rightarrow 9$, puis ins.	$3 + 1 = 4$
5	9	4	Ins. directe	1
6	9	5	Ins. directe	1
7	9	6	Ins. directe	1
8	9	7	Ins. directe	1
9	9	8	Ins. directe	1
10	9	9 (plein)	Resize $9 \rightarrow 27$, puis ins.	$9 + 1 = 10$
Coût total				23

Vérification : $1 + 2 + 1 + 4 + 1 + 1 + 1 + 1 + 1 + 10 = 23$. ✓

Coût total pour 10 insertions : **23**.

Question 2 :

L'idée est de séparer le coût :

- Chaque insertion contribue 1 à chaque tour (coût de base).
- Les resizes ajoutent des coûts supplémentaires ponctuels.

Ces coûts supplémentaires forment une série géométrique de raison 3, dont la somme est bornée par $3N/2$.

Correction détaillée

Pour N insertions, le nombre de resizes j vérifie $3^j \leq N < 3^{j+1}$, soit $j = \lfloor \log_3 N \rfloor$.

Décomposition du coût :

$$T(N) = \underbrace{N}_{\substack{\text{coût de base} \\ (1 \text{ par insertion})}} + \underbrace{\sum_{i=0}^j 3^i}_{\substack{\text{coûts de copie} \\ \text{des resizes successifs}}}$$

Calcul de la somme géométrique :

$$\sum_{i=0}^j 3^i = \frac{3^{j+1} - 1}{2} \leq \frac{3N - 1}{2} \quad (\text{car } 3^j \leq N \Rightarrow 3^{j+1} \leq 3N)$$

Borne supérieure :

$$T(N) \leq N + \frac{3N}{2} = \frac{5N}{2}$$

Vérification sur 10 insertions : $j = \lfloor \log_3 10 \rfloor = 2$ (car $3^2 = 9 \leq 10 < 27 = 3^3$). $T(10) = 10 + (1 + 3 + 9) = 10 + 13 = 23$. ✓

Question 3 :

L'analyse amortie demande : « sur N insertions, combien *chacune* coûte-t-elle en moyenne ? » Si le total est $O(N)$, alors la moyenne est $O(N)/N = O(1)$: chaque insertion coûte une quantité constante *en moyenne*, même si ponctuellement un resize entraîne un coût plus grand.

Correction détaillée

Méthode de l'agrégat : on a montré $T(N) \leq \frac{5N}{2}$. Donc :

$$\text{Coût amorti par insertion} = \frac{T(N)}{N} \leq \frac{5N/2}{N} = \frac{5}{2} = O(1).$$

Interprétation :

- *Coût pire cas individuel* : lors d'un resize sur une capacité c , le coût est $c + 1 = O(c)$, potentiellement grand.
- *Coût amorti* : cette situation est rare (un resize à capacité c n'arrive qu'après $c - 1$ insertions à coût 1). Le coût du resize se « dilue » sur les $c - 1$ insertions précédentes.
- On peut le penser ainsi : chaque insertion « paie » pour elle-même (coût 1) et « met de côté » une petite réserve pour le futur resize.